

Personal Project System Design
Document
Minor AI for Society

Casey Stijlaart

Table of Contents

1. Document history	1
2. Introduction	2
3. System Description	3
3.1. Extensions	3
4. System Design	5
4.1. Components	5
4.2. Use Cases	7
4.3. Requirements	9
4.4. Requirements Matrix	12
5. System behavior	13
5.1. Sequence Diagrams	13
5.2. Class Diagrams : ESP Firmware	20
5.3. Flow Charts : ESP Firmware	23
6. Hardware Diagram	27

1. Document history

Table 1. Document History

Version	Date	Status	Author	Description	Remarks
0.1	30-03-2026	Draft	Casey Stijlaart	Initial Draft	-
0.1	01-04-2026	Draft	Casey Stijlaart	Additional Details and design	-
1.0	08-04-2026	First Version	Casey Stijlaart	Completed	-
1.1	10-06-2026	Updated	Casey Stijlaart	Revision. Updated various parts to reflect latest software	-

2. Introduction

This document serves as a comprehensive guide detailing the design and architecture of the embedded system that focusses on plant monitoring and care. The system is designed to monitor various environmental parameters such as soil moisture, temperature, humidity, and light intensity to ensure optimal conditions for plant health. It contains a detailed system description with multiple diagrams defining all of the hardware components and the behavior of the system.

The behavioral diagrams in this document (class diagrams, flow charts, and sequence diagrams) are generated from the firmware source code (v1.2) and therefore describe the system as realized. A complete API reference of the firmware, generated with Doxygen from the annotated source code, accompanies this document at <related/html/index.html>.

3. System Description

The plant health monitoring system is an embedded solution designed to monitor environmental conditions and provide feedback on plant health. The system is centered around an ESP32 microcontroller, which collects data from sensors for soil moisture, temperature, humidity, and light level.

The ESP32 processes this data locally: each monitoring cycle builds a feature vector from the recent sensor history, runs a lightweight machine learning model to classify the plant's health status (healthy, moderate stress, high stress), predicts the time remaining until watering is needed, and generates concrete care recommendations (water, reduce temperature, increase light). Results are stored locally on the device and uploaded to a cloud backend (Supabase), from which a mobile app presents them to the user.

The system follows a modular architecture consisting of a sensor array (data collection), the ESP32 edge node (processing and ML inference), a cloud data storage (central source of truth), and a mobile user interface (visualization and configuration). New devices are paired through BLE provisioning from the app, after which the edge node operates autonomously: it runs a scheduled monitoring cycle every three hours and polls the cloud every five seconds for user-issued commands such as an on-demand measurement or a factory reset.

3.1. Extensions

The following extensions are categorized using the MoSCoW prioritization method to define which features are essential, desirable, optional, or out of scope for the current version of the plant health monitoring system.

3.1.1. Must

These are the core extensions or capabilities required for the system to meet its primary objective.

- The system must monitor key environmental factors relevant to plant health: soil moisture, temperature, humidity, and light level.
- The system must process sensor data and provide plant health feedback to the user.
- The system must present the measured values and plant status through a user interface.
- The system must operate on the ESP32 platform as the main embedded controller.
- The system must support a basic warning or alert when plant conditions fall outside acceptable ranges.
- The system must classify the plant condition as healthy, moderate stress, or high stress.

3.1.2. Should

These features are important and significantly improve the system.

- The system should allow the user to enter plant-specific preferences, such as whether a plant prefers low, medium, or high moisture.
- The system should store historical sensor readings for later review, both locally on the device and in the cloud.
- The system should support multiple plants and multiple edge nodes per user.
- The system should provide personalized recommendations for plant care actions, including a prediction of when watering will next be needed.
- The system should estimate an optimized measurement interval and the resulting power saving based on the predicted plant needs.

3.1.3. Could

These are optional enhancements that may improve usability or functionality if time and resources allow.

- The system could include additional sensors, such as air quality sensors.
- The system could provide trend visualization, such as graphs of moisture or temperature over time.
- The system could automatically apply the optimized measurement interval instead of only reporting it.

3.1.4. Won't

These items are intentionally excluded from the current scope of the project.

- The system won't include full automation of plant care, such as automatic watering, in the current version.
- The system won't support large-scale commercial greenhouse management.
- The system won't include advanced computer vision or camera-based plant disease detection.
- The system won't aim to support every plant species with fully customized biological models.

4. System Design

4.1. Components

The system is composed of several components that work together to monitor plant health and provide feedback to the user. To outline each component and their role, an overview is given, see [Table 2](#). Next to that, a component diagram, [Figure 1](#) is given to illustrate the relationships between the components.

Table 2. Component Description

Component	Description
Edge Node (EN)	The Edge Node is the ESP32 firmware. It reads the sensor array, performs feature engineering and local ML inference, generates care recommendations, and keeps a local history on flash storage (LittleFS). It uploads measurements and model metrics to the data storage, polls it for user commands, and is configured through BLE provisioning.
Sensor Array (SA)	The sensor array consists of the sensors that monitor the plant's environment: a capacitive soil moisture sensor, a DHT22 temperature/humidity sensor, and an LDR light sensor. These sensors provide real-time data to the Edge Node.
User Interface (UI)	The user interface is a mobile app that allows the user to manage plant profiles, view plant health status and historical data, and configure devices. It communicates with the data storage for data access and uses BLE to provision nearby Edge Nodes.
Data Storage (DS)	A Supabase backend that stores device registrations (devices table), plant profiles and settings (plant_settings), sensor readings (plant_readings), model metrics (model_metrics), and device logs (logs). Acts as the central source of truth for the system and enables remote access to plant data.

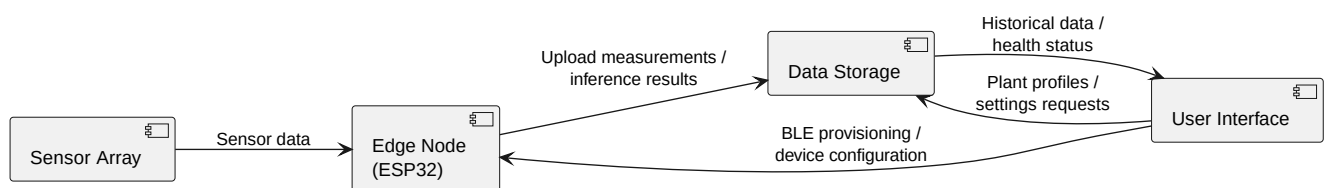


Figure 1. Component Diagram

4.1.1. Messages Between Components

To showcase how each component is connected to another, the main messages that connect them are listed, see [\[Messages between components\]](#). The message names correspond to the methods in the implemented software; supporting calls (heartbeats, logging, device deletion) are omitted here and can be found in the sequence diagrams and the API reference.

Table 3. Messages between components

Message name	From	To	Description
<code>ReadPercent()</code> , <code>Read()</code> , <code>ReadLightLevelPct()</code>	Edge Node	Sensor Array	Reads the soil moisture, temperature/humidity, and light level.
<code>RegisterDevice(deviceId, deviceName)</code>	Edge Node	Data Storage	Registers the device after successful provisioning.
<code>SendReading(json)</code> / <code>SendModelMetrics(json)</code>	Edge Node	Data Storage	Uploads a monitoring cycle result and its model metrics.
<code>FetchProfileSettings(plantLabel)</code>	Edge Node	Data Storage	Retrieves the assigned plant profile (thresholds and preferences).
<code>CheckTriggerMeasurement()</code> / <code>CheckTriggerReset()</code>	Edge Node	Data Storage	Polls the user command flags every 5 seconds.
<code>insert/update plant_settings</code>	Interface	Data Storage	Stores a new or updated plant profile.
<code>query plant_readings</code>	Interface	Data Storage	Requests current and historical plant data for visualization.
<code>set trigger_measurement</code> / <code>trigger_reset</code>	Interface	Data Storage	Requests an on-demand measurement or removal of a device.
BLE provisioning (write credentials)	Interface	Edge Node	Discovers an unpaired Edge Node and sends WiFi credentials and cloud configuration during setup.

4.2. Use Cases

This section describes the main interactions between the user and the system. The use cases cover device pairing, plant monitoring, configuration, data visualization, and device removal, as realized in the implemented system.

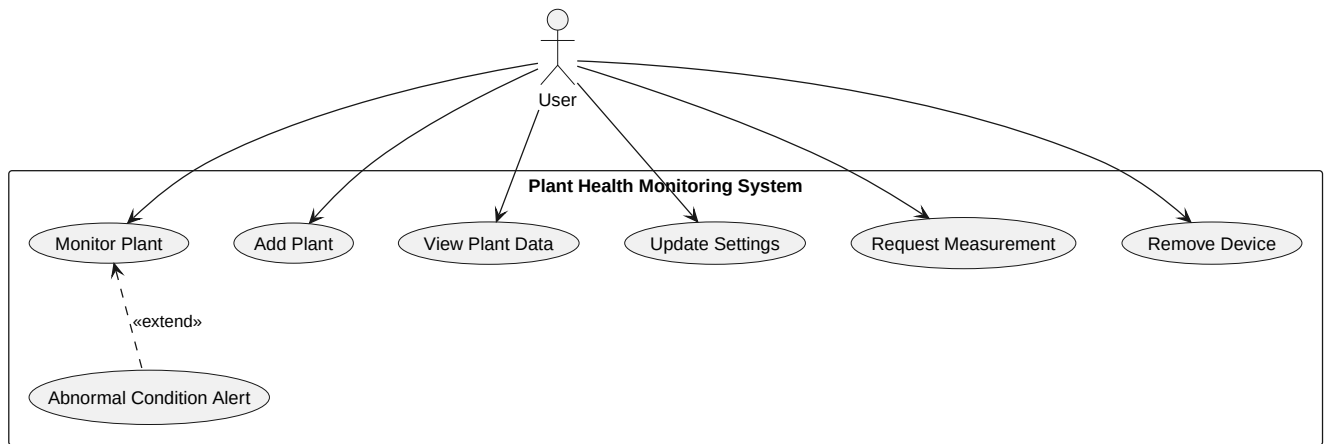


Figure 2. Use Case Diagram

[Use Case Diagram] depicts the use cases. It shows the system boundary and specifies how the actor interacts with the system. Each use case is described in the following tables.

Table 4. Use Case 01

ID: UC01	Use Case Name: Add Plant
Description	The user adds a new plant by pairing a device over BLE through the app, then saves the plant name and care preferences, which the device picks up to start monitoring.
Actor(s)	User

Table 5. Use Case 02

ID: UC02	Use Case Name: Monitor Plant
Description	The system monitors plant conditions on a fixed schedule using sensor data and on-device ML inference, and makes the results available to the user.
Actor(s)	User

Table 6. Use Case 03

ID: UC03	Use Case Name: View Plant Data
Description	The user views current and historical plant data, health status, and care recommendations through the app.
Actor(s)	User

Table 7. Use Case 04

ID: UC04	Use Case Name: Update Settings
Description	The user updates plant care settings such as moisture, temperature, humidity, and light preferences. The device applies the updated profile before each scheduled cycle.
Actor(s)	User

Table 8. Use Case 05

ID: UC05	Use Case Name: Abnormal Condition Alert
Description	The system detects abnormal plant conditions during monitoring and notifies the user with the stress level and suggested actions.
Actor(s)	User

Table 9. Use Case 06

ID: UC06	Use Case Name: Request Measurement
Description	The user requests an immediate measurement from the app, which the device answers with a full monitoring cycle.
Actor(s)	User

Table 10. Use Case 07

ID: UC07	Use Case Name: Remove Device
Description	The user removes a device from the app. The device performs a factory reset, erasing all local and cloud state, and reboots into pairing mode.
Actor(s)	User

4.3. Requirements

A system has various requirements, each one covering an important aspect of how the system must work or behave. These requirements are divided into the user-, functional- and non-functional requirements. They reflect the system as realized in firmware v1.1.0.

4.3.1. User Requirements

Table 11. User Requirements

ID	Requirement	Use Case(s)
UR_001	The user should be able to add a new plant by pairing a device and entering its care requirements.	UC01
UR_002	The user should be able to monitor the health status of a plant.	UC02, UC03
UR_003	The user should be able to view current and historical plant data.	UC03
UR_004	The user should be able to update plant care settings.	UC04
UR_005	The user should receive notifications and care recommendations when abnormal conditions occur.	UC05
UR_006	The user should be able to request an immediate measurement on demand.	UC06
UR_007	The user should be able to remove a device so it can be re-paired.	UC07

4.3.2. Functional Requirements

Table 12. Functional Requirements

ID	Requirement	Use Case(s)
FR_001	The user interface must allow the user to input plant details and preferences when adding a plant.	UC01
FR_002	The data storage must store device registrations, plant profiles, and associated care settings.	UC01, UC04
FR_003	The edge node must support BLE provisioning to receive WiFi credentials and cloud configuration from the user interface.	UC01
FR_004	The edge node must register itself in the data storage after provisioning and persist its pairing state locally.	UC01
FR_005	The sensor array must provide environmental data: soil moisture, temperature, humidity, and light level.	UC02
FR_006	The edge node must read sensor data and derive a feature vector from the recent measurement history.	UC02
FR_007	The edge node must use an on-device ML model to classify plant health and predict the time until watering is needed.	UC02, UC05
FR_008	The edge node must store measurement results locally and send sensor data, predictions, and model metrics to the data storage.	UC02
FR_009	The data storage must provide plant data when requested by the user interface.	UC03
FR_010	The user interface must display current and historical plant data.	UC03
FR_011	The user interface must allow the user to update plant care settings, and the edge node must fetch the updated profile before each scheduled cycle.	UC04
FR_012	The edge node must detect abnormal conditions and generate care recommendations based on sensor data and predictions.	UC05
FR_013	The user interface must notify the user when abnormal conditions are detected.	UC05
FR_014	The edge node must poll the data storage for command flags and run an immediate monitoring cycle when a measurement is requested.	UC06
FR_015	The edge node must perform a full factory reset (clear history, pairing state, cloud registration, and device storage) when removal is requested.	UC07

4.3.3. Non-Functional Requirements

Table 13. Non-Functional Requirements

ID	Requirement
NFR_001	The system should be modular so components can be updated or replaced independently.
NFR_002	The system should support multiple edge nodes and plants.
NFR_003	The edge node must operate within embedded hardware constraints such as limited memory and processing power (ML inference, BLE, and TLS combined on a single ESP32).
NFR_004	WiFi credentials and the cloud API key must not be stored in the firmware binary; they are transferred once over BLE during provisioning, and removing a device must erase them.
NFR_005	The edge node should operate autonomously after provisioning, without requiring the user interface to be active.

4.4. Requirements Matrix

The requirements matrix provides a consolidated view of all the requirements, showing how they relate to each other and to the use cases. This matrix helps ensure that all requirements are covered and that there are no gaps in the system design.

Table 14. Use Cases vs User Requirements Coverage

	UR_001	UR_002	UR_003	UR_004	UR_005	UR_006	UR_007
UC01	X						
UC02		X					
UC03		X	X				
UC04				X			
UC05					X		
UC06						X	
UC07							X

Table 15. Functional Requirements vs User Requirements Coverage

	UR_001	UR_002	UR_003	UR_004	UR_005	UR_006	UR_007
FR_001	X						
FR_002	X			X			
FR_003	X						
FR_004	X						
FR_005		X					
FR_006		X					
FR_007		X			X		
FR_008		X					
FR_009			X				
FR_010			X				
FR_011				X			
FR_012					X		
FR_013					X		
FR_014						X	
FR_015							X

5. System behavior

5.1. Sequence Diagrams

Per use case, there is an associated sequence diagram that describes the interactions between the system components to achieve the desired functionality. The diagrams are generated from the firmware source code (v1.1.0) and show the realized interactions between the app, the firmware classes, and the cloud backend.

5.1.1. UC01 : Add Plant

Scenario: A user adds a new plant by pairing a device through the app. On first boot (unpaired) the device advertises a BLE service (PHM-XXXXXX) that the app uses to transfer the WiFi credentials, API key, and cloud URL. After a successful WiFi connection the device registers itself in the **devices** table and writes the pairing document to flash; the app then detects the new device, lets the user save the plant name and preferences, and the device picks up the plant assignment through its 5-second polling (see UC06). If the WiFi connection fails, the device wipes its state and restarts so the user can retry.

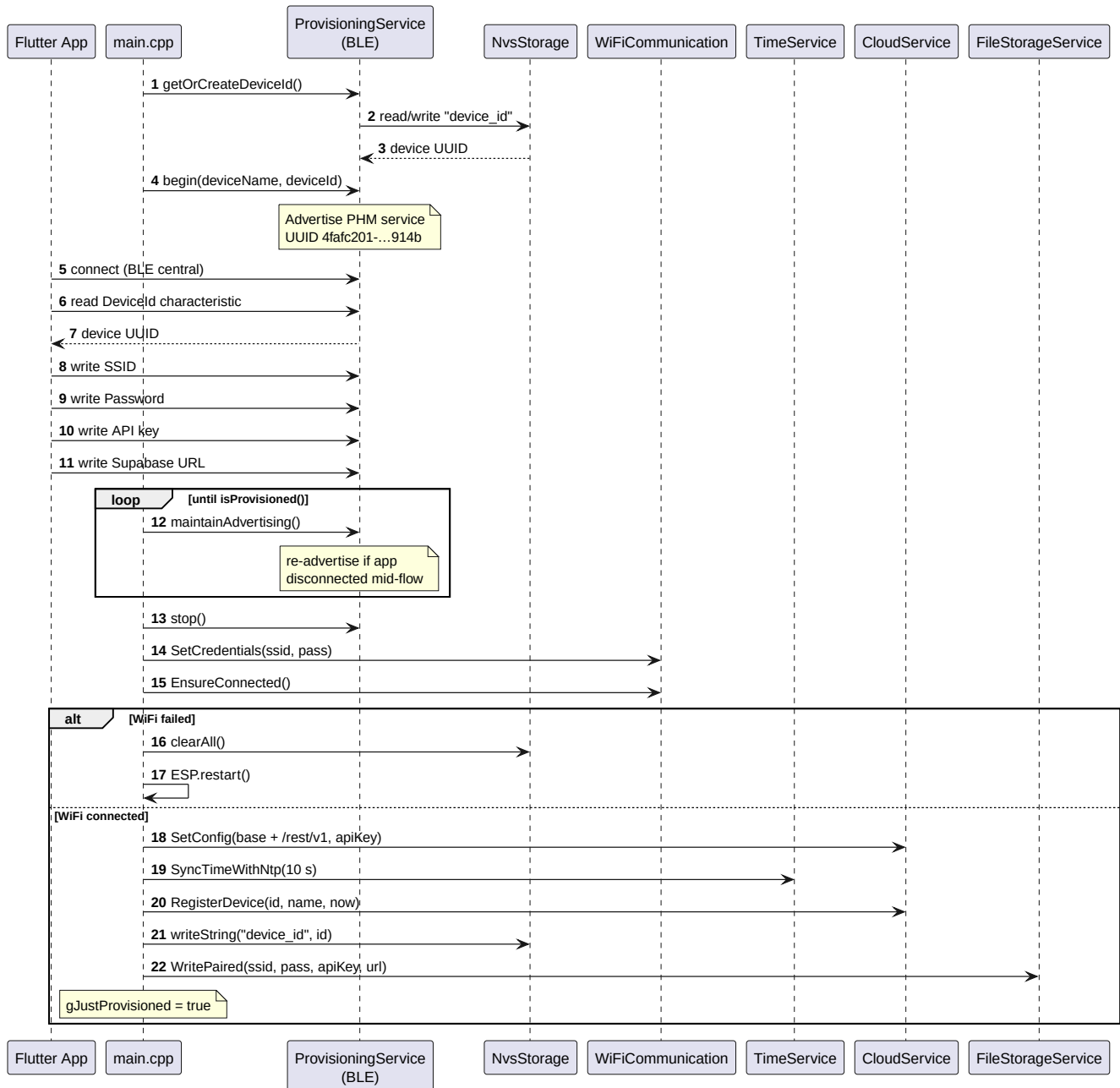


Figure 3. UC01 Sequence Diagram — BLE provisioning and device registration

5.1.2. UC02 : Monitor Plant

Scenario: The system monitors the plant's conditions on a schedule (every 3 hours). Each cycle reads all sensors, stores the snapshot in the history buffer, builds the feature vector, and runs the ML inference and irrigation prediction.

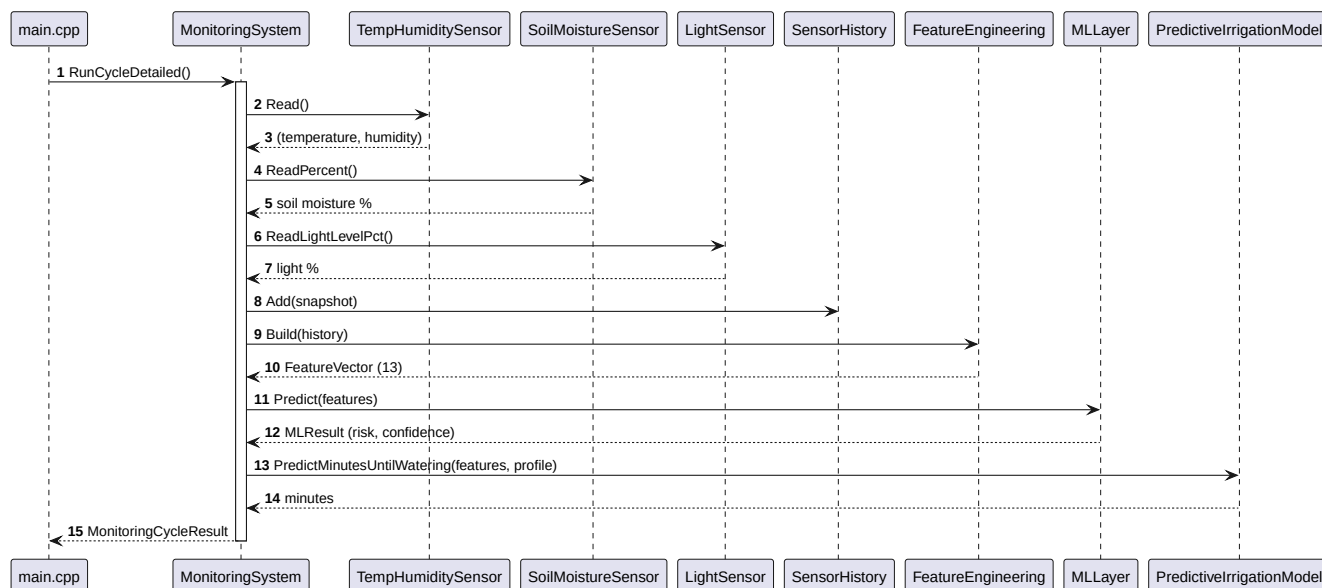


Figure 4. UC02 Sequence Diagram — monitoring cycle

5.1.3. UC05 : Abnormal Condition Alert

Scenario: At the end of each monitoring cycle, the recommendation engine translates the prediction into care actions. The result is appended to the local history file; if a plant is assigned, the reading and model metrics are also uploaded to the cloud, where the app picks them up and notifies the user of abnormal conditions and suggested actions.

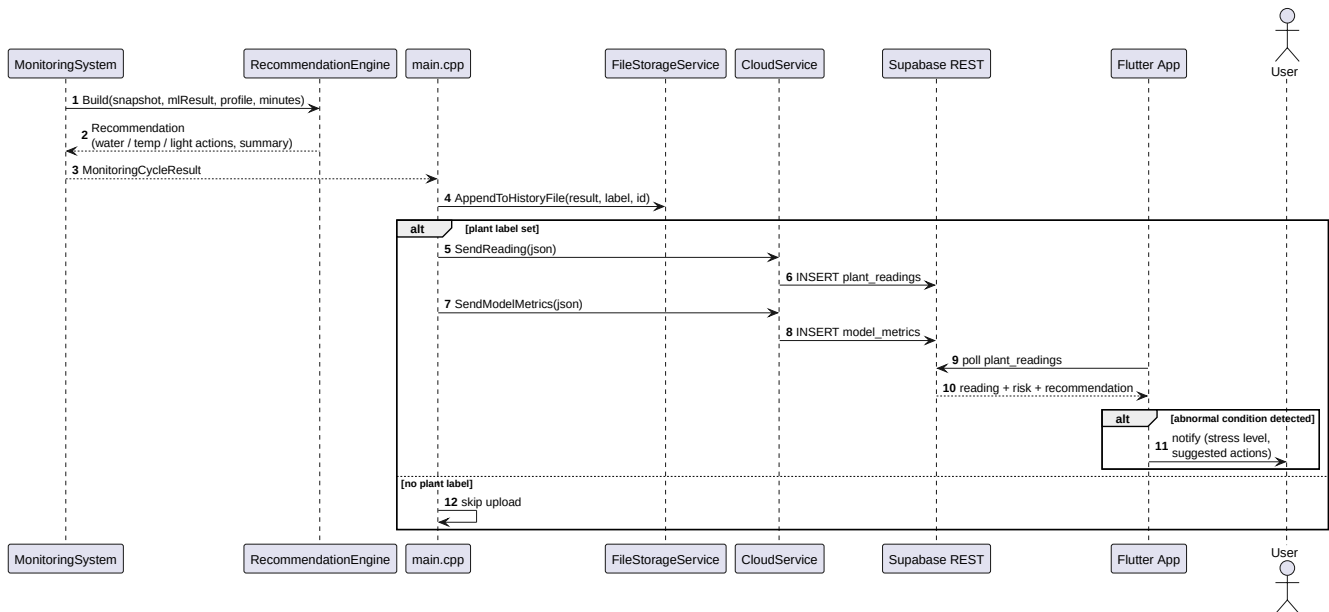


Figure 5. UC05 Sequence Diagram — recommendation, cloud upload + alert

5.1.4. UC03 + UC04 : View Plant Data + Update Settings

Scenario: The user views plant data and changes settings through the app, which reads from and writes to the data storage directly (**plant_readings**, **plant_settings**). No interaction with the device is required to view data; the device applies updated settings by re-fetching its plant profile (**FetchProfileSettings**) before every scheduled monitoring cycle.

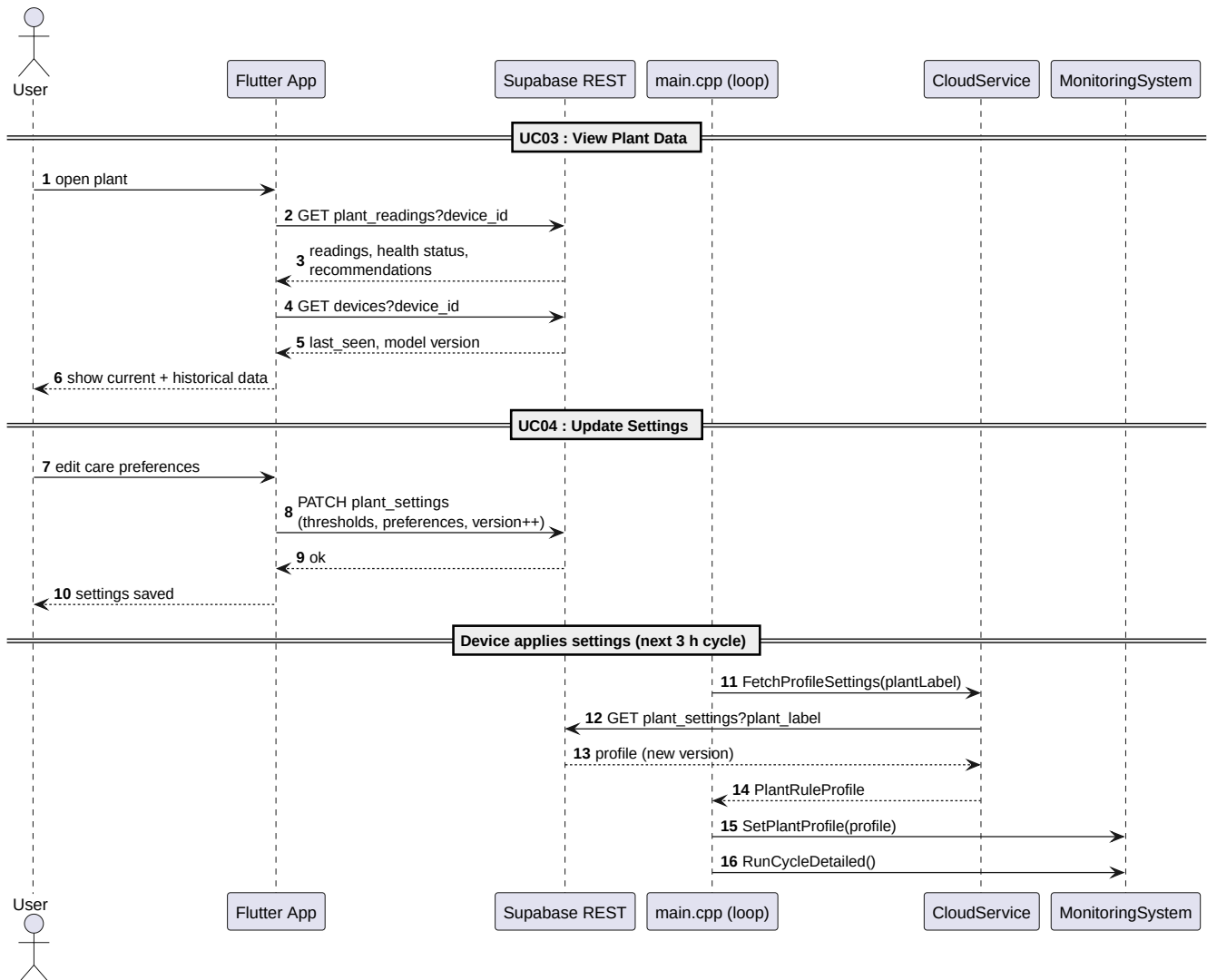


Figure 6. UC03 + UC04 Sequence Diagram — view plant data + update settings

5.1.5. UC06 : Request Measurement

Scenario: The user requests an immediate measurement from the app, which sets the `trigger_measurement` flag on the device's row. Every 5 seconds the device polls the command flags, answers a set measurement flag with a full monitoring cycle, and flushes any queued log lines.

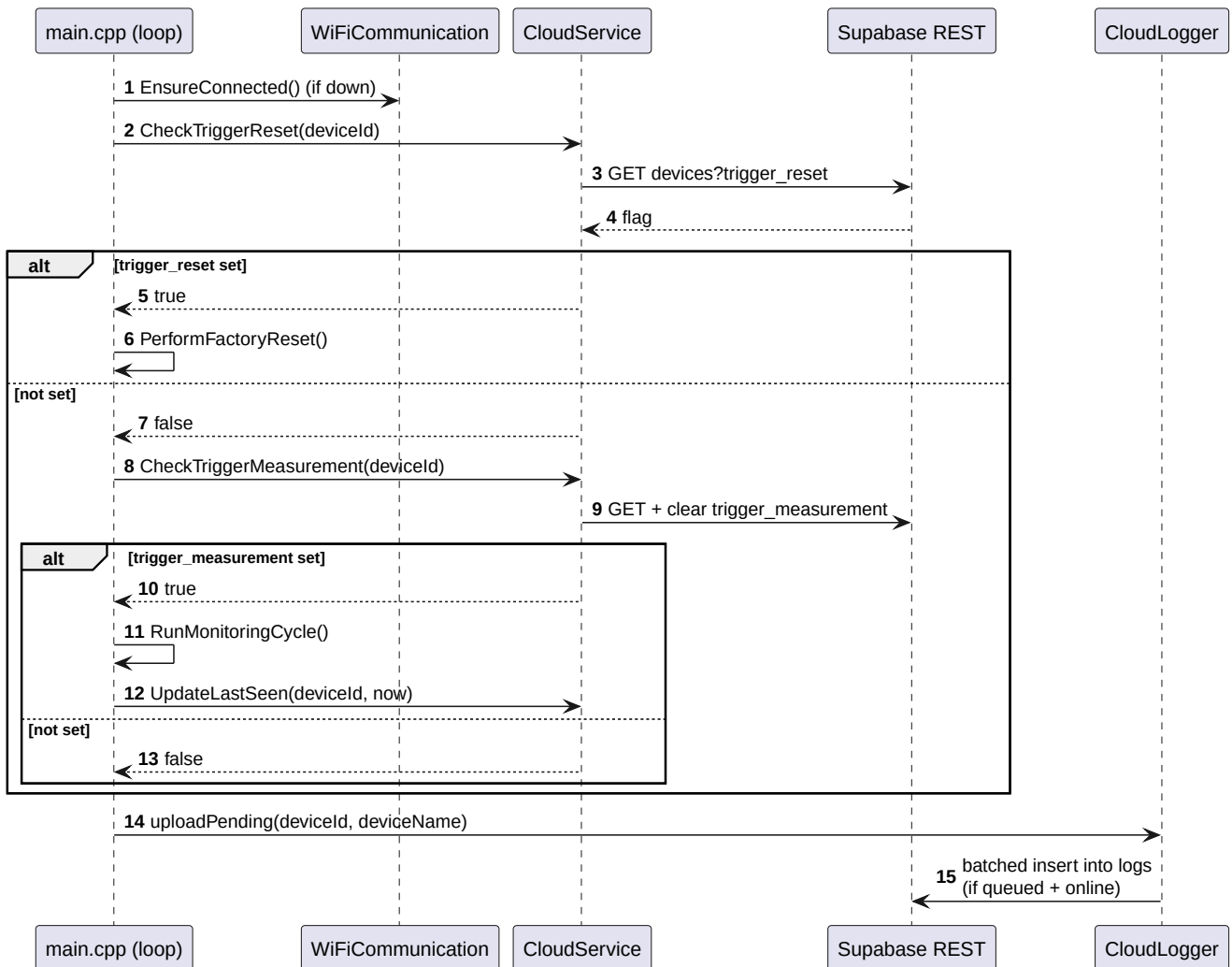


Figure 7. UC06 Sequence Diagram — cloud command polling (every 5 s)

5.1.6. UC07 : Remove Device

Scenario: The user removes a device from the app, which sets the `trigger_reset` flag (the same reset also runs when a plant assignment is missing beyond the grace period). The device uploads pending logs, clears its local history and pairing state (erasing the WiFi credentials), deletes its registration from the `devices` table, wipes its non-volatile storage, and restarts into BLE provisioning mode so it can be paired again.

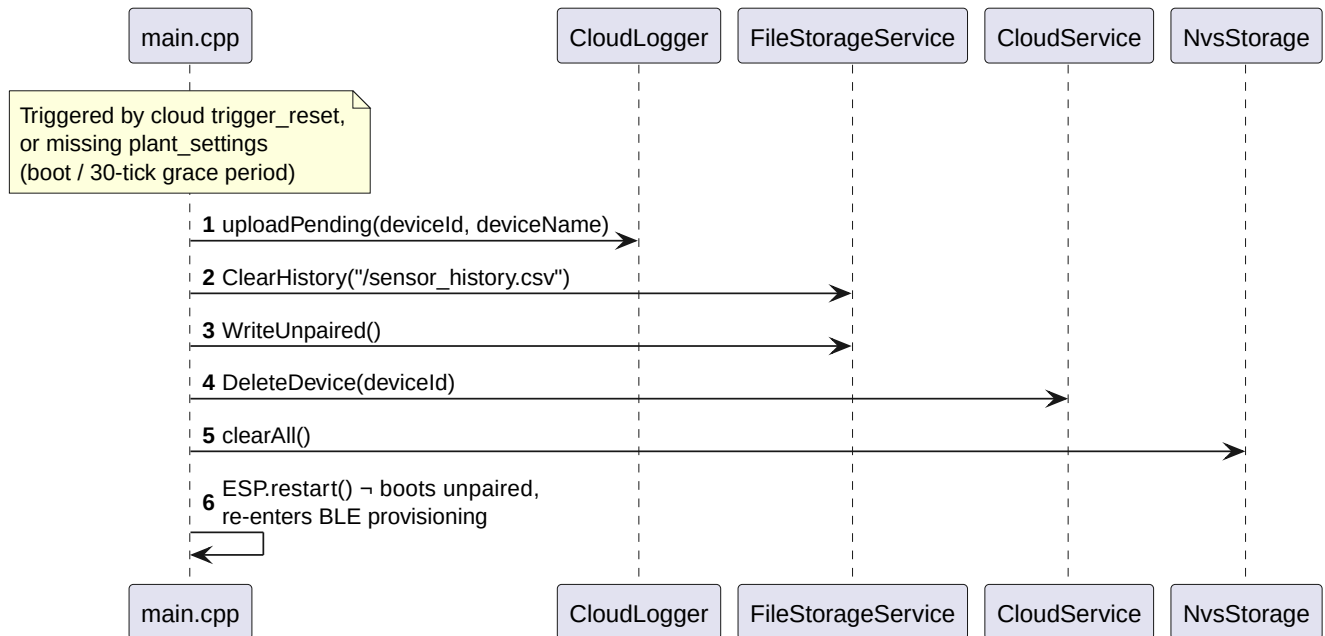


Figure 8. UC07 Sequence Diagram — factory reset

5.2. Class Diagrams : ESP Firmware

The class diagrams below illustrate the structure and relationships between the classes that make up the ESP32 firmware (v1.1.0). They are generated from the source and header files in `firmware/src` and `firmware/include`, and are split into three logical groups: the monitoring pipeline, the shared data types, and the connectivity, storage, and service classes.

5.2.1. Monitoring pipeline

The `MonitoringSystem` class orchestrates a full measurement cycle: it reads the soil moisture, temperature/humidity, and light sensors, stores the snapshot in a ring buffer (`SensorHistory`), builds a 13-value feature vector (`FeatureEngineering`), runs the ML inference (`MLLayer`) and the predictive irrigation model, and finally asks the `RecommendationEngine` to translate the results into user-facing actions.

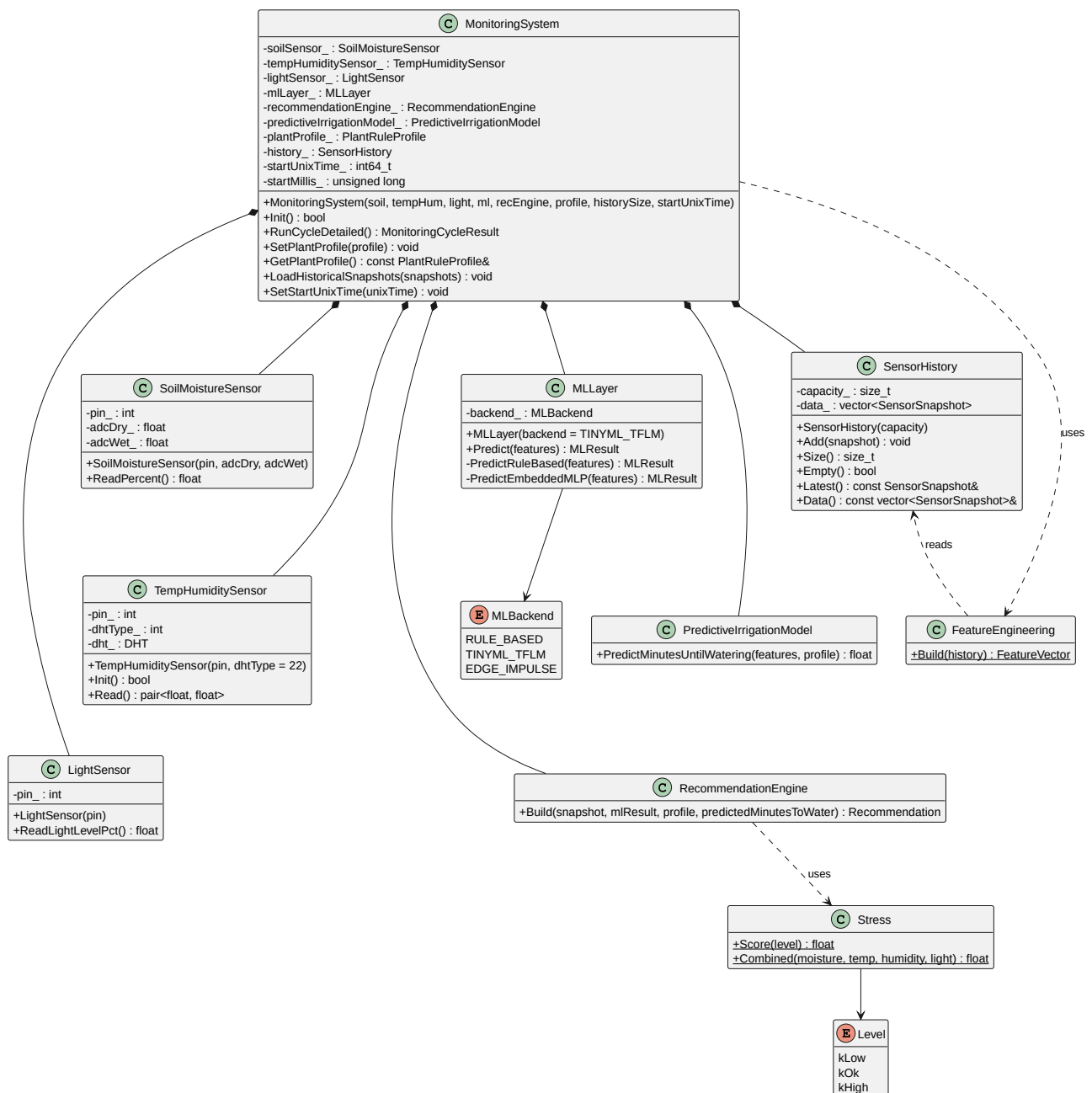


Figure 9. Class Diagram — monitoring pipeline (sensors → ML → recommendation)

5.2.2. Shared data types

All components exchange data through a small set of plain structs defined in `PlantTypes.hpp`. A `MonitoringCycleResult` bundles the raw snapshot, the feature vector, the ML result, and the recommendation produced in one cycle. The `PlantRuleProfile` holds the per-plant thresholds and user preferences fetched from the cloud.

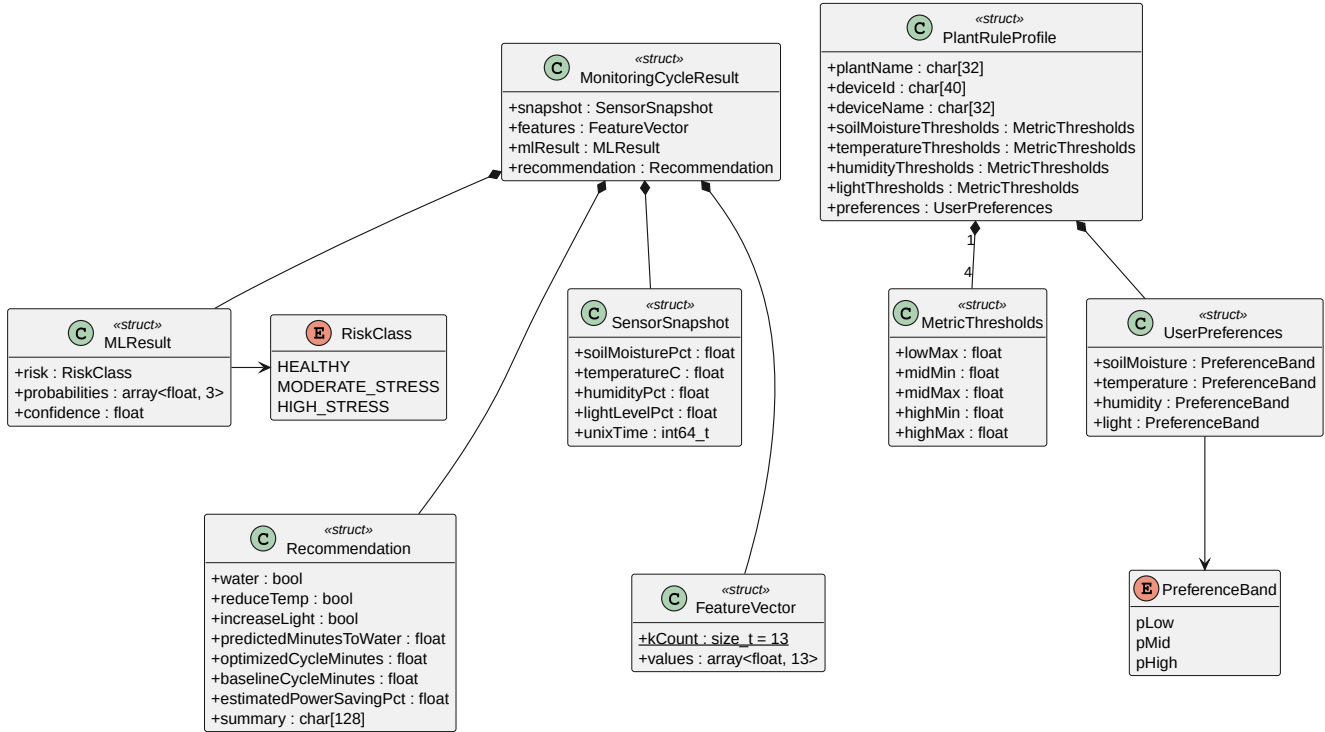


Figure 10. Class Diagram — shared data types (`PlantTypes.hpp`)

5.2.3. Connectivity, storage & services

The service layer connects the device to the outside world: **WiFiCommunication** manages the network link, **CloudService** talks to the Supabase REST API, **ProvisioningService** handles BLE pairing on first boot, and **FileStorageService/NvsStorage** persist history and pairing state locally. **CloudLogger** mirrors serial log output to the cloud, and **main.cpp** orchestrates all of them.

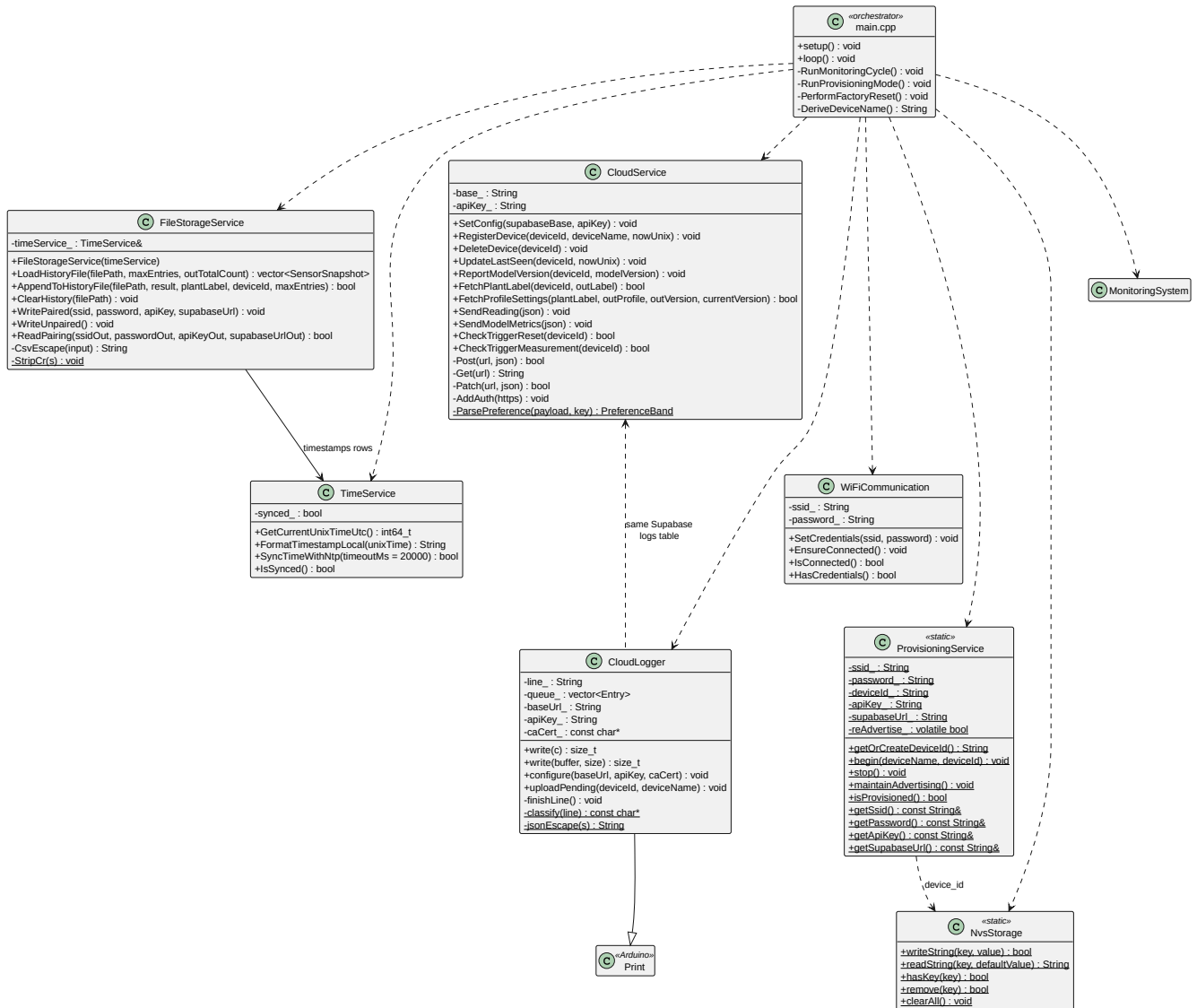


Figure 11. Class Diagram — connectivity, storage & services

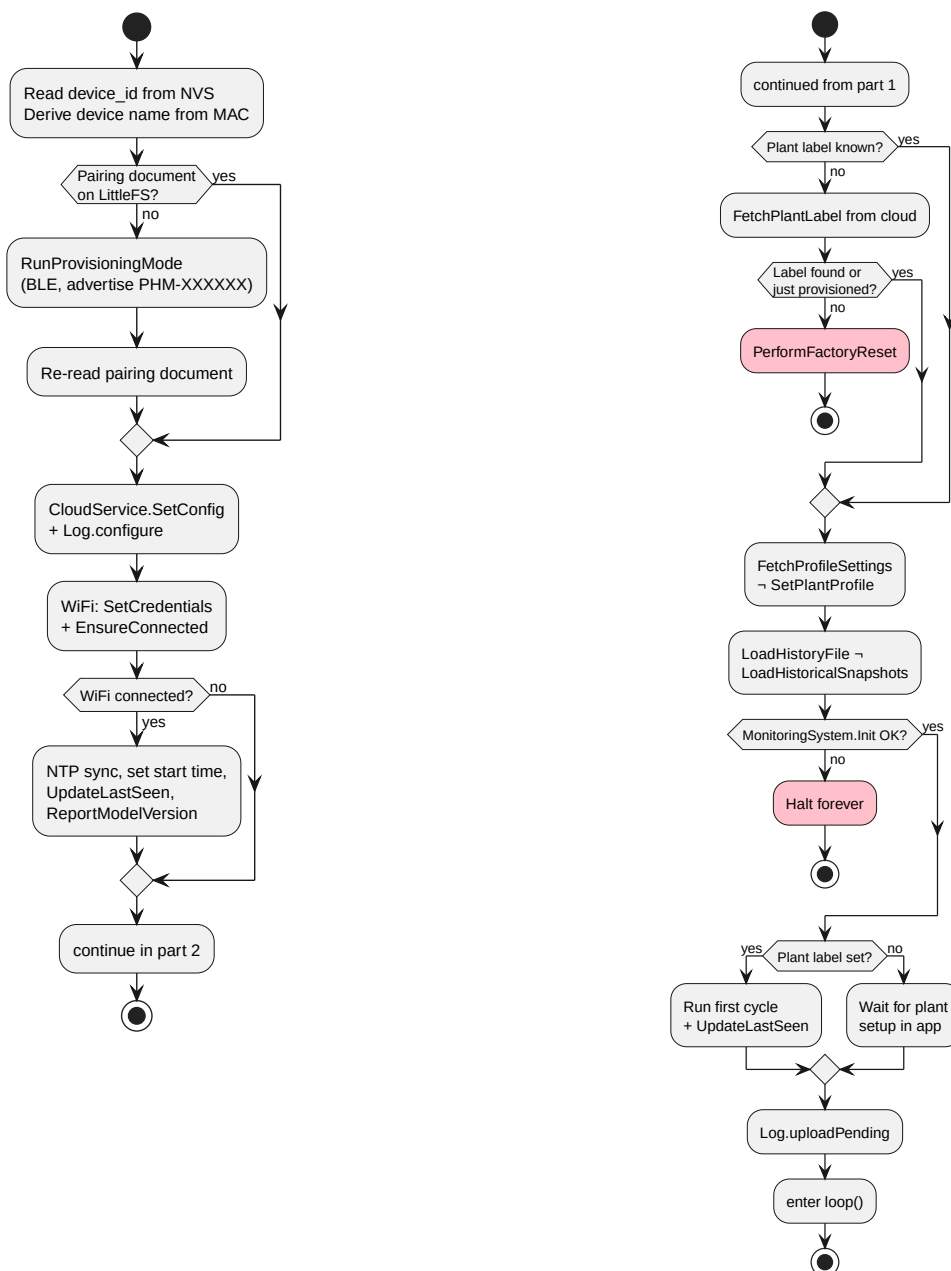
5.3. Flow Charts : ESP Firmware

The flow charts below describe the control flow of the firmware: the boot sequence, the main loop, and the internals of a single monitoring cycle. The longer flows are split into parts that are shown next to each other.

5.3.1. Boot flow

On boot the firmware decides between provisioning mode (no pairing document on flash) and normal operation (part 1). Once connected, it fetches the plant label and profile from the cloud, restores the sensor history, and runs the first monitoring cycle (part 2). A missing plant assignment outside the provisioning grace period triggers a factory reset.

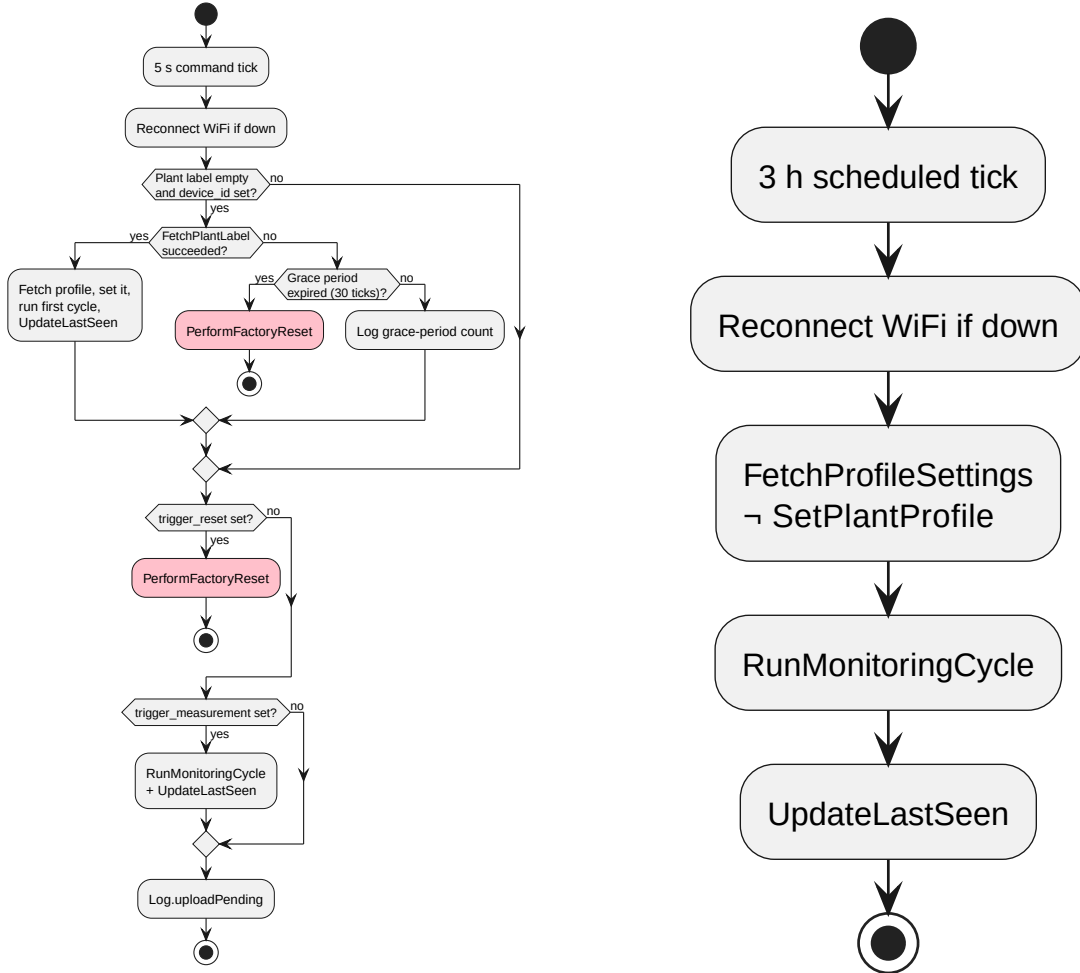
Table 16. Flow Chart — boot / `setup()` flow (part 1: provisioning + connectivity, part 2: plant profile + first cycle)



5.3.2. Main loop

The main loop runs two timers: a 5-second tick that polls the cloud for commands (`trigger_reset`, `trigger_measurement`) and watches for a plant being assigned, and a 3-hour tick that refreshes the plant profile and runs a scheduled monitoring cycle.

Table 17. Flow Chart — main `loop()` flow (left: 5-second command tick, right: 3-hour scheduled tick)



5.3.3. Monitoring cycle internals

A single cycle reads all sensors, validates the soil reading, appends the snapshot to the history buffer, builds the feature vector, runs the ML prediction and irrigation prediction in parallel, and produces the final recommendation.

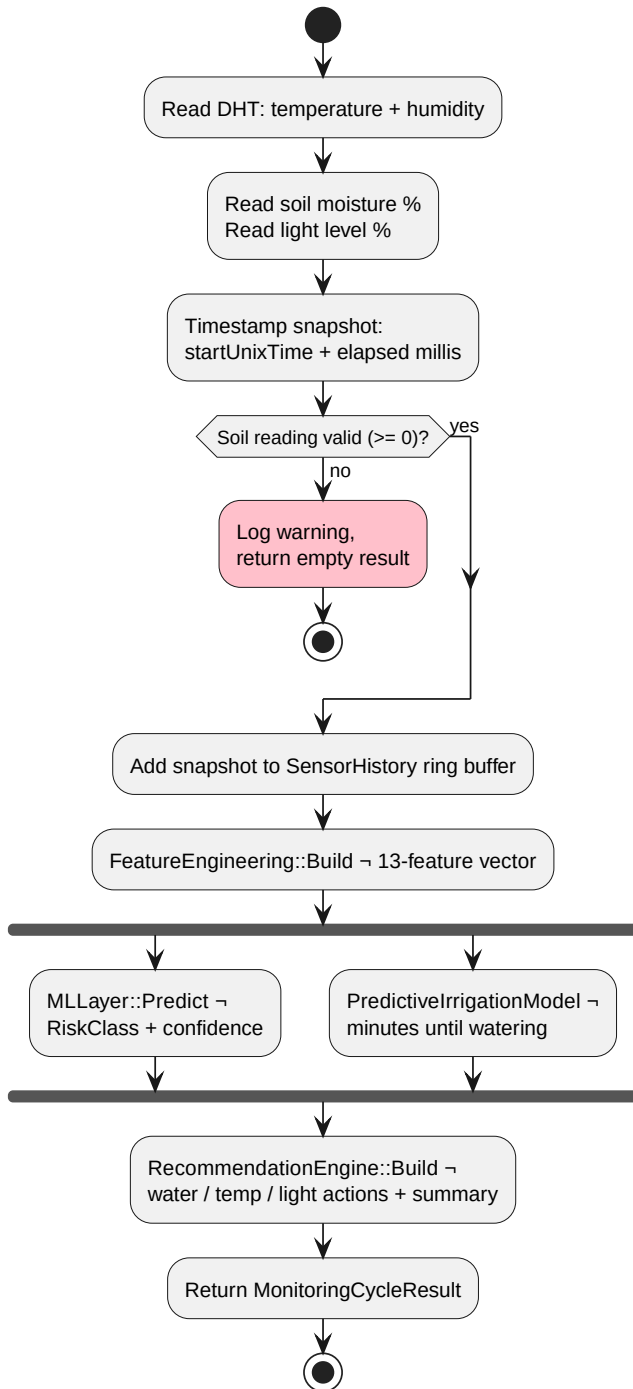


Figure 12. Flow Chart — monitoring cycle (*MonitoringSystem::RunCycleDetailed*)

5.3.4. Pairing states and timing constants

The behavior in the flow charts is driven by the pairing state stored on flash (`/pairing_state.txt`) and a small set of timing constants. WiFi credentials are only persisted in the pairing document; marking the device unpaired erases them by construction, so an unpaired device retains no WiFi secrets.

Table 18. Pairing states

State	Behaviour
File missing	Created as <code>unpaired</code> on first read → BLE provisioning mode
<code>unpaired</code>	BLE provisioning mode — wait for app
<code>paired</code> + plant settings exist	Normal operation
<code>paired</code> + no plant settings (boot)	Immediate reset → BLE provisioning mode
<code>paired</code> + no plant settings (running)	Grace period → reset → BLE provisioning mode

Table 19. Key firmware constants

Constant	Value	Purpose
<code>kIntervalMs</code>	3 hours	Regular monitoring cycle interval
<code>kCommandCheckMs</code>	5 seconds	Command flag poll interval
<code>kHistorySize</code>	56 entries	Local sensor history ring buffer
<code>kMaxNoPlantTicks</code>	30 ticks (2.5 min)	Grace period before reset when no plant is assigned

6. Hardware Diagram

The system is composed of several hardware modules. The table, [\[Used hardware components\]](#), describes the individual hardware components. The diagram, [\[Hardware connection\]](#), shows the connections between the various hardware components, including the GPIO pin assignments used by the firmware.

Table 20. Used hardware components

Hardware block	Description
ESP32 DevKit	Runs the ML models, processes the sensor data, and stores the pairing state and sensor history on flash (LittleFS). Uses the 3 MB huge_app partition table to fit BLE, TinyML, and TLS.
Capacitive soil moisture sensor	Analog, GPIO 34. Calibrated from ADC 3500 (dry) to 1450 (wet).
DHT22	Digital temperature and humidity sensor, GPIO 4.
LDR light sensor	Analog, GPIO 35, with a 10 k Ω pull-down resistor.
Phone (mobile app)	Displays data and recommendations, pairs devices over BLE, and issues commands through the cloud.
USB power supply	Powers the ESP32 over USB (5 V).

[\[\[Hardware connection\]\]](#) .Hardware connection image::images/hw-diagram.svg[]